

HW4 Programming: Imitation Learning *[50 points]*

CS/STAT 184(0): Introduction to Reinforcement Learning

(Due November 26 at 11:59 pm ET)

In this problem, you will implement two fundamental imitation learning algorithms – behavioral cloning and DAgger – using the PyTorch machine learning framework.

1 Requirements

This question uses Python 3.9 or higher. The required libraries are found in `environment.yml`. **You may not use any additional packages.**

You may install the requirements using a package manager of your choice. We recommend [micromamba](#), a *much* faster implementation of [conda](#). Create a conda environment named `184-hw4` with the required packages using the following terminal commands:

- `micromamba create -f environment.yml`
- `micromamba activate 184-hw4`

2 Background and Setting

You will implement two fundamental imitation learning algorithms: **behavioral cloning (BC)** and **dataset aggregation (DAgger)**. We will use the CartPole and MountainCar environments from OpenAI Gym. These both have finite action spaces.

The goal in imitation learning is as follows: given some form of “access” to an unknown expert policy $\pi^\star$, we want to learn a policy $\hat{\pi}$ that is at least as good as the expert.

- **Behavioral cloning** is the simplest form of imitation learning: we get access to a dataset of N state-action pairs $\mathcal{D}^\star = \{s_i^\star, a_i^\star\}_{i=1}^N$ from the expert $\pi^\star$. Then we perform *supervised learning* this dataset. More formally, given a family of policies Π to search over, we aim to find the policy that *maximizes the likelihood* of the dataset’s state-action pairs:

$$\hat{\pi} = \operatorname{argmax}_{\pi \in \Pi} \frac{1}{N} \sum_{(s^\star, a^\star) \in \mathcal{D}^\star} \log \pi(a^\star | s^\star).$$

This is equivalent to *minimizing the cross entropy loss* between the actual actions $a^\star$ in the expert dataset and the actions predicted by our learner π . BC is an entirely *offline* imitation learning algorithm that does not require interaction with the environment.

- **Dagger** is similar to behavioral cloning but assumes that we can *query* the expert π^* . After obtaining a trajectory from the current policy, we can ask what actions the expert *would* have taken in the *states* of that trajectory, obtaining a set of “expert” state-action pairs $\{s_h, \pi^*(s_h)\}_{h=0}^{H-1}$. We can then “aggregate” this rollout into the dataset $\mathcal{D}^*$ and learn from all the experiences collected so far using behavioral cloning.

You will implement these algorithms in the following files:

- `learner.py`: Functions defining the imitation learning agents.
- `train.py`: Full training script.

3 Implementing Imitation Learning Agents

If you are unfamiliar with PyTorch, we recommend reviewing how its [optimizers](#) and [loss functions](#) work. The specifications for the following functions can be found in their docstrings in the source code.

Q1 [10 points] Implement the `ImitationLerner.learn` method in `learner.py`.

Q2 [10 points] Implement the `Dagger.sample_from_logits` and `Dagger.rollout` methods in `learner.py`.

4 Training Loop

Q [10 points] Implement the `train_dagger` and `train_bc` functions in `train.py`.

5 Training

Q1 [5 points] Run `train.py` to train policies, using the `--env` argument to select between CartPole and MountainCar and the `--algo` argument to select between BC and DAgger. This will generate plots in the `plots` directory of the training losses across epochs.

Once you have trained an agent, you can **evaluate** it by running `test.py` with the corresponding arguments. Pass the `--render` flag to visualize the agent’s performance.

We have *intentionally omitted* default values for the algorithm hyperparameters (run `train.py -h` for a list). You should experiment with different hyperparameters to achieve good performance. (Tip: in most terminals, you can use the up and down arrow keys to navigate through your command history.) You may first want to figure out how many expert samples are given in the dataset.

Q2 [5 points] In `README.md`, for each of the four algorithm-environment combinations, report the hyperparameters you used and the mean reward achieved. Both BC and DAgger should be able to solve the CartPole and MountainCar environments (i.e. consistently achieve 500 and (at least) -120 total reward respectively).

Q3 [10 points] Briefly discuss the differences between the four configurations. Why did you choose these hyperparameters? Your answer should be a few sentences long.

6 Deliverables

Please submit a .zip file with directory tree like the following. Your solution should modify only the starred files.

```
YOUR_HUID/
├── dataset.py
├── environment.yml
├── expert.py
├── learner.py (*)
├── README.md (*)
├── test.py
├── train.py (*)
├── utils.py
├── plots (*)
├── bc_CartPole-v1.png
├── bc_MountainCar-v0.png
├── dagger_CartPole-v1.png
├── dagger_MountainCar-v0.png
├── expert_policies
│   ├── CartPole-v1_policy.pt
│   └── MountainCar-v0_policy.pt
├── learned_policies (*)
│   ├── bc_CartPole-v1.pt
│   ├── bc_MountainCar-v0.pt
│   ├── dagger_CartPole-v1.pt
│   └── dagger_MountainCar-v0.pt
├── expert_data
│   ├── CartPole-v1_dataset.pt
│   └── MountainCar-v0_dataset.pt
```